

Information Gain

Measures how much a split **reduces entropy**.

$$IG = Entropy(parent) - \sum \frac{N_{child}}{N_{parent}} Entropy(child)$$

Higher Information Gain → better split.

Gini Impurity

Another measure of impurity, commonly used in **CART**.

$$Gini = 1 - \sum p_i^2$$

- Gini = **0** → completely pure
- Lower Gini → better purity

Gini is generally faster to calculate than Entropy because it doesn't use logarithms.

4. Numerical Features

For numerical features, the tree:

1. Sorts the values.
2. Tries different **thresholds**.
3. Calculates impurity/Information Gain for each split.
4. Selects the **best threshold**.

Example:

Age: 20 25 30 35 40

Possible split:

Age < 27.5

Age < 32.5

Age < 37.5

...

The split producing the **best impurity reduction** is selected.

5. Categorical Features

Decision Trees can also split using categorical features such as:

- Gender
- Occupation
- City
- Type of product

Advantages

- Easy to understand and visualize
- Handles numerical and categorical features
- Requires little preprocessing
- No feature scaling is usually required
- Can capture non-linear relationships

Disadvantages

- Can easily **overfit**
- Sensitive to small changes in data
- Deep trees can become very complex

Remember

Decision Tree = Nested if-else conditions

Entropy/Gini → measure impurity

Information Gain → choose the best split

Leaf → final prediction

Decision Tree Hyperparameters — Soft Copy Notes

Overfitting

Model learns **training data too closely**, including noise.

- Training accuracy → high
- Test accuracy → low

Underfitting

Model is **too simple** to learn the actual pattern.

- Training accuracy → low
- Test accuracy → low

Important Hyperparameters

1. max_depth

Controls the **maximum depth of the tree**.

- High → complex tree → **overfitting**
- Low → simple tree → **underfitting**

2. min_samples_split

Minimum number of samples required to **split a node**.

- Low → more splits → overfitting
- High → fewer splits → prevents overfitting

3. min_samples_leaf

Minimum number of samples required in a **leaf node**.

- Low → smaller leaves → overfitting
- High → larger leaves → simpler tree

4. max_features

Maximum number of features considered when finding a split.

- Lower value → less complexity → can reduce overfitting
- Higher value → more features considered → more complex tree

5. criterion

Determines how the quality of a split is measured.

Common options:

- **Gini**
- **Entropy**
- **Log Loss** (depending on implementation/version)

Main Goal

Tune hyperparameters to find the balance between overfitting and underfitting.

Easy memory:

max_depth → How deep?

min_samples_split → When can I split?

min_samples_leaf → How small can a leaf be?

max_features → How many features to consider?

criterion → How good is the split?

Regression Tree — Paste-Ready Notes

1. What is a Regression Tree?

- A **Regression Tree** is a Decision Tree used to predict a **continuous numerical value**.
- Example: predicting **house price, salary, temperature, sales**, etc.
- Classification Tree → predicts **class/label**.
- Regression Tree → predicts **number**.
- At a leaf node, prediction is usually the **mean of target values** in that leaf.

2. How does it work?

1. Start with the complete dataset.
2. Try different **features and split points**.
3. Calculate the error for each possible split.
4. Select the split that gives the **lowest error**.
5. Repeat this process recursively for each child node.
6. Stop according to conditions such as max_depth, min_samples_leaf, etc.

3. Splitting Criterion — MSE

For regression, **Mean Squared Error (MSE)** is commonly used:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$$

Where:

- y_i = actual value
- $\bar{y}$ = mean value of the node
- Lower MSE → **better/purer split**

The algorithm chooses the split that produces the **lowest weighted MSE** among the child nodes.

4. Prediction

Suppose a leaf contains:

₹20L, ₹22L, ₹24L

Prediction:

$$\frac{20 + 22 + 24}{3} = 22L$$

So the tree predicts **₹22 lakh** for a new sample falling into that leaf.

5. Important Hyperparameters

Hyperparameter Meaning

criterion	Measures split quality, commonly squared_error (MSE)
max_depth	Maximum depth of the tree
min_samples_split	Minimum samples needed to split a node
min_samples_leaf	Minimum samples required in a leaf
max_features	Number of features considered for each split

Overfitting control:

- High max_depth → more complex → may overfit

- Low max_depth → simpler → may underfit
- Higher min_samples_leaf → generally reduces overfitting

6. Advantages

- Easy to understand.
- No feature scaling required.
- Captures **non-linear relationships**.
- Can show **feature importance**.
- Works well for regression problems.

7. Practical Implementation

from sklearn.tree import DecisionTreeRegressor

```
model = DecisionTreeRegressor(
    criterion="squared_error",
    max_depth=5,
    min_samples_leaf=5
)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

Use **cross-validation** to get a more reliable estimate of model performance and to tune hyperparameters.

★ Remember

Regression Tree = Decision Tree + Continuous Numerical Prediction

Classification Tree → classes

Regression Tree → numbers

Regression split → minimize MSE/error

Leaf prediction → mean of target values

100

dtreeviz — Short Notes

What is dtreeviz?

dtreeviz is a Python library used to create **clear and informative visualizations of Decision Trees**.

It is more detailed and easier to understand than the basic sklearn tree plot.

Why use dtreeviz?

Standard sklearn visualization can:

- Become cluttered for large trees.
- Make splits difficult to understand.
- Show limited information about data distribution.

Key Features

1. Visualizes Splits

- Clearly shows which feature and threshold are used at each split.

2. Shows Data Distribution

- Displays **histograms/scatter plots** inside nodes.
- Helps understand how data is divided.

3. Prediction Path

- Shows the path followed by a particular data point from **root** → **leaf**.
- Useful for understanding why the tree made a prediction.

4. Customization

- Can show/hide node information.
- Supports different layouts such as horizontal trees.
- Can simplify the visualization using options like fancy=False.

5. Decision Boundary

- Can visualize how a tree divides the feature space.

6. Feature Importance

- Helps visualize which features contribute most to predictions.

★ Main Purpose

dtreeviz = Better visualization + easier interpretation + debugging of Decision Trees

It is especially useful when you want to **explain how a Decision Tree makes its predictions.**
